

Workshop: Late Deliveries

Technologies for Artificial Intelligence — two hours in class, after lesson 5

Fabio Antonini — Università degli Studi dell'Aquila

Date to be announced · not examinable

Contents

What this is	1
The files	1
The dataset	2
The plan	2
What each step should produce	2
The mistakes the checks catch	3
The answers to the six questions	3
The rule, for the reveal	3

What this is

Two extra hours, held in class after lesson 5, in which the students take one small dataset through the whole workflow of lessons 1 to 5 while the instructor guides from the projector. It adds no new material: everything it uses has been taught. Its purpose is practice, and a quick, honest signal to each student of what they can already do on their own.

It is **not examinable** and nothing is collected. Each step ends with a one-sentence question whose answer goes in a markdown cell, because explaining a choice is what the exam asks for.

The files

File	What it is
<code>workshop.ipynb</code>	the students' notebook: six steps, each with blanks written <code>___</code> , a check cell and a question
<code>checks.py</code>	the checks: each prints a tick, or a cross with a hint, by comparing the student's result with a reference computed from the same data, split and folds
<code>delivery_data.py</code>	the data generator; it holds the rule, so students should not open it before the end
<code>workshop_slides.pptx</code>	a title and five slides, to open and close the session

The notebook runs in the course container like every other: open `Workshop/workshop.ipynb` in JupyterLab. No network is needed.

The dataset

1,200 parcels from a courier company. Six columns - `distance_km`, `parcels_on_van`, `driver_years`, `departure_hour`, `rain_mm` (about one reading in ten missing, at random), `vehicle_age_years` - and the target `late`. 31% of the parcels are late, so always answering *on time* scores 0.690.

The plan

Time	Minutes	Segment
0:00–0:10	10	Slides 2–4: how the session works; everyone opens the notebook and runs the first cell
0:10–0:22	12	Step 1: look at the data, compute the baseline
0:22–0:34	12	Step 2: the stratified split
0:34–0:50	16	Step 3: imputer, scaler and logistic regression in one pipeline
0:50–1:05	15	Step 4: 5-fold cross-validation on the training set
1:05–1:25	20	Step 5: the late share by hour, then squared terms on the same folds
1:25–1:37	12	Step 6: the chosen model, tested once
1:37–2:00	23	Slides 5–6: show of hands per step, the rule revealed, discussion
120		

A rhythm that works: give the instruction, leave the room a few minutes, then show the check passing on the projector before moving on, so that nobody falls two steps behind.

What each step should produce

These are the numbers the checks expect, computed in the course container.

Step	Result
1	baseline 0.690; <code>rain_mm</code> has 126 missing values
2	900 parcels to fit, 300 to test, 31.0% late in both
3	the pipeline <code>SimpleImputer</code> → <code>StandardScaler</code> → <code>LogisticRegression</code>
4	cross-validated accuracy 0.762 ± 0.020
5	with squared terms, 0.858 ± 0.024 on the same folds

Step	Result
6	test accuracy 0.873, against a cross-validated 0.858

The mistakes the checks catch

Each is a mistake the course has warned about, so a cross is a cue to point back to the lesson that covered it.

Mistake	Step	Back to
the share of late parcels given as the baseline, instead of the share on time	1	lesson 1
a split without <code>stratify=y</code>	2	lesson 1
the gaps filled in <code>X</code> before the split, outside the pipeline	3	lesson 2
cross-validation run on the whole dataset, test set included	4	lesson 5
the straight model taken to the test set although step 5 preferred the curved one	6	lesson 5

The answers to the six questions

1. `rain_mm` has 126 gaps, about 10%. The baseline comes first because a score means nothing until you know what the trivial answer scores: here 0.690, so a model at 0.70 has learned almost nothing.
2. With a different share of late parcels the two parts would be measuring different problems, and the test score would move for reasons that have nothing to do with the model. With 300 test parcels a few points of difference in the share is already enough to shift the accuracy visibly.
3. Statistics learned from the whole table - the median of `rain_mm`, the mean and standard deviation of every column - would include the test rows, which is leakage: the test set would no longer be unseen. Inside the pipeline they are learned from the training part of each fold only.
4. Yes: 0.762 against 0.690, seven points. The $\pm$ is the standard deviation across the five folds; a difference much smaller than it is not a difference you can trust.
5. 0.858 against 0.762 is about four times the $\pm$ of either, so the gain is real. The squared terms helped `departure_hour`: lateness is high early and late in the day and low around midday, a U that a straight line cannot draw.
6. 0.873 against 0.858 ± 0.024 : within the spread, so the cross-validation told the truth. A test score far above the cross-validated one would be a warning, most often of leakage or of a test set that is too small or unrepresentative.

The rule, for the reveal

The generator draws the probability of a late parcel from

$$\log \frac{p}{1-p} = -9.6 + 0.1575(d - 30) + 0.07(v - 90) - 0.315(e - 8) + 0.44(h - 13)^2 + 0.70r$$

with d the distance in km, v the parcels on the van, e the driver's years, h the departure hour and r the rain in mm. In words: longer routes, fuller vans and rain make a parcel later, experience makes it earlier, and departure time works through rush hour - the further from 13:00, the later, in both directions. **vehicle_age_years has no effect at all**, which is the answer to the optional exercise at the end of the notebook.

Two things worth saying at the reveal. The logistic regression with squared terms is exactly the shape of the rule, which is why it wins; on real data nobody hands you the rule, and the cross-validation of step 5 is how you would have found out. And the remaining 13% of errors is not a failure of the model: the rule gives probabilities, not certainties, so even a perfect model gets some parcels wrong.

A worked solution for the instructor is kept out of the repository, like the solutions to the exercises.